第一章　多变量异常检测算法

1.1　多变量时间序列的归一化方法

1.1.1　RevIN 的主要思想

单个多变量时间时间序列为一个行向量，训练时以及推理时的输入为多个时间序列，因此为矩阵。下面简述归一化思想，具体符号以及计算下文将详细介绍。

传统的 Z-Score 或 Min-Max 归一化方法在处理非平稳（Non-stationary）时间序列（即均值和方差随时间变化）时往往会失效，这一问题已在 ICLR / ICML 等顶级会议中被反复讨论。本文使用 Reversible Instance Normalization (RevIN，ICLR 2022) [1]。原因分析如下：CESN 依赖于储备池状态 r 和嵌入 z 的几何特性（模长）。如果输入数据的整体均值发生偏移（Concept Drift），会导致：状态 r 饱和（tanh 接近 ± 1），或嵌入 z 的模长整体漂移；从而破坏基于模长的异常检测机制。

RevIN 的主要思想：

- 归一化 Normalize： 输入数据的归一化，将所有数据归一化至 $[-1,1]$ 区间内。在输入网络前，计算当前时间窗口的均值 μ 和标准差 σ，对输入进行标准化：

$$x' = \frac{x-\mu}{\sigma}. \tag{1.1}$$

  注意到行向量为输入数据，因此对于输入矩阵，计算列项量的均值和方差，然后根据公式 (1.1) 作用于该列的每个输入数据。

- Forward： 网络以 x' 作为输入进行处理。

- 去归一化 Denormalize： 网络输出 $\hat{y}'$ 后，应用逆变换：

$$\hat{y} = \hat{y}' \cdot \sigma + \mu. \tag{1.2}$$

RevIN 作为我们方法的补充，可视为原输入前和输出后又分别加了一层分别进行归一化和去归一化处理。我们可以在 Normalize 之后的数据上计算嵌入 z 和模长 r。此时，模长主要反映的是**局部形态**（Local Pattern）**的强度**，而不是全局数值尺度的大小。同时符合随机嵌入方法中，与几何特征假设的建模思想保持一致。

1.1.2 输入数据以及归一化处理

在多变换量时间序列异常检测中，我们定义：

- 原始多变量数据流：$U = \{u_1, u_2, \ldots, u_t, \ldots\}$，其中 $u_t \in \mathbb{R}^{N_{var}}$ 表示 t 时刻的 N_{var} 个传感器读数。
- 模型输入（时间窗口）：为了捕捉时间依赖性，模型的输入是长度为 L 的滑动窗口矩阵。记 t 时刻的输入矩阵为 $X_t \in \mathbb{R}^{L \times N_{var}}$: $X_t = [u_{t-L+1}, u_{t-L+2}, \ldots, u_t]^T$
- 预测目标：模型通常预测下一个时刻的值 $y_{target} = u_{t+1} \in \mathbb{R}^{N_{var}}$。

RevIN 旨在解决时间序列的非平稳问题（即统计特性随时间漂移），这对 ESN 的储备池稳定性至关重要。它包含归一化 (Normalize) 和反归一化 (Denormalize) 两个过程，并引入可学习的仿射变换参数。

参数定义：$\gamma, \beta \in \mathbb{R}^{N_{var}}$: 可学习的缩放和平移参数 (Affine Parameters)。它们是网络参数 Θ 的一部分，随反向传播更新。

1.1.3 归一化算法过程

1. 统计量计算 (Instance Statistics):

对输入窗口 X_t 的每一列 (每个变量) 计算均值和标准差。

$$\mu_t = \frac{1}{L} \sum_{i=1}^{L} X_t[i,:] \quad (\in \mathbb{R}^{N_{var}})$$

$$\sigma_t = \sqrt{\frac{1}{L}\sum_{i=1}^{L}(X_t[i,:]-\mu_t)^2+\epsilon} \quad (\in \mathbb{R}^{N_{var}})$$

其中 ϵ 为防止除零的小值 (如 1e -5)。

2. 归一化 (Normalization):

$$\hat{X}_t = \frac{X_t-\mu_t}{\sigma_t}$$

注意: 这里的减法和除法是利用广播机制 (Broadcasting) 按变量维度进行的。

3. 可学习仿射变换 (Learnable Affine): 为了保留一定的分布信息, 应用可学习参数:

$$\tilde{X}_t = \hat{X}_t \odot \gamma + \beta$$

$\tilde{X}_t$ 时为 CESN 网络的核心输入。

4. 反归一化 (Denormalization):

假设网络输出的归一化预测值为 $\tilde{y}_{pred} \in \mathbb{R}^{N_{var}}$。为了计算物理空间的误差或进行可视化，我们需要将其还原。

- 逆仿射: $\hat{y}_{pred} = (\tilde{y}_{pred} - \beta) \oslash \gamma$

- 逆统计: $y_{final} = \hat{y}_{pred} \odot \sigma_t + \mu_t$

1.2　网络相关参数

1. 输入层 (Input Layer): 输入层的输入为归一化后的列向量。注意到 ESN 的特性，储备池用于捕捉序列数据间的关系。将多变量序列数据逐个输入，基于数据更新储备池的状态 r。为了捕捉动态，我们假设 CESN 逐步读取输入内的 L 个向量，取最后一个状态。

$$r_k = (1-\delta)r_{k-1} + \delta\tanh(W_{in}\tilde{X}_t[k] + W_{res}r_{k-1}), \quad k = 1\ldots L.$$

令最终状态 $h_{res} = r_L \in \mathbb{R}^{N_{res}}$。注意到，与单变量的符号表示不同（原来输入状态是 r），本文的输入状态都记为 h_{res}.

储备池 $r(t)$ 通常为 $500-1000$ 维度。

δ 称为泄漏率 (Leaking Rate)，来源于泄漏积分型回声状态网络 (Leaky-Integrator ESN)。δ 的物理意义：它控制储备池状态更新的速度（即记忆的长短）。如果 $\delta \approx 1$：状态几乎完全被当前的输入和新的内部连接更新，记忆遗忘得很快。如果 $\delta \approx 0$：状态变化非常缓慢，大部分保留上一时刻的值，记忆保持时间长（适合低频信号）。它相当于一个低通滤波器。δ 作为一个超参数 (Hyperparameter) 给定的，并且在训练过程中是固定的（例如取 $\delta \in [0.1, 1.0]$）。我们可以通过网格搜索或验证集来选择最好的 δ。保持 δ 固定有助于维持储备池的回声状态属性，即保证网络的稳定性。

3. MLP 投影头 (Non-linear Projection): 将储备池状态映射到适合几何分离的空间。在上下文语境清晰的情况下，可以省略 t。

$h_{mlp} = \mathrm{ReLU}(W_{mlp1} h_{res} + b_1)$, $z_{raw}(t) = W_{mlp2} h_{mlp} + b_2$ (这里$z_{raw} \in \mathbb{R}^{N_{emb}}$)。当上下文清晰时，$z_{raw}(t)$ 简记为 z。

4. 几何归一化 (Geometry):

$r(t) = ||z_{raw}(t)||_2$

$\alpha(t) = \frac{z_{raw}(t)}{r(t)}$ (投射到单位超球面)。其中，嵌入向量 $\alpha(t)$ 是一个压缩的共享表示（Shared Representation）。当上下文清楚时，省略符号 t。

输入为 h_{res}，$h_{res} \to h_{mlp} \to z \to r \to \alpha$(归一化后) $\to \tilde{y}_{pred}$

5. 随机读出层 (Stochastic Readout): 使用 DropConnect 生成掩码矩阵 $M \sim \mathrm{Bernoulli}(p)$，其中 $M_{ij} \in \{0, 1\}$。预测输出:

$$\tilde{y}_{\text{pred}} = (W_{\text{out}} \odot M)\alpha$$

注意：为了训练稳定性，我们的 Loss 计算在归一化空间进行。此时的目标值为 $\tilde{y}_{\text{target}} = \mathrm{Affine}(\mathrm{Norm}(u_{t+1}))$。

M 是一个矩阵，其形状（即行、列长度）与它所作用的权重矩阵 W_{out}

完全一致。假设 W_{out} 的维度是 $N_y \times N_{\text{emb}}$（输出维度 × 嵌入维度），那么 $M \in \{0,1\}^{N_y \times N_{\text{emb}}}$。

DropConnect 不需要进行训练，它是应用在训练过程中的一种正则化或增强手段。过程：在每次前向传播 (Forward Pass) 时，我们根据概率 p 随机生成一个由 0 和 1 组成的矩阵 M。$M_{ij} = 1$ 的概率为 $1-p$（保留权重）。$M_{ij} = 0$ 的概率为 p（丢弃权重）。实际用于计算的权重是

$$\tilde{W} = W_{\text{out}} \odot M \quad \text{(元素对应相乘)}$$

DropConnect 迫使 W_{out} 的每一个权重分量都必须足够鲁棒，不能依赖于特定的神经元组合。

1.3　损失函数定义及其含义

定义:

$$\mathscr{L}(\Theta) = \frac{1}{B}\sum_{i=1}^{B} \mathbb{E}_M \left[1 - \exp\left(-\frac{\|\tilde{y}_{target}^{(i)} - (W_{out} \odot M)\alpha^{(i)}\|^2}{2\sigma^2} \right) \right] + \lambda_1 \|W_{out}\|^2 + \lambda_2 \|\Theta_{\text{MLP}}\|^2$$

解释:

1. **预测目标为 $u(t+1)$ 归一化后的值, 即上述公式中的 $\tilde{y}_{target}^{(i)}$。** 异常通常表现为违反预测规律。如果我们能够准确预测下一步，说明系统处在正常轨道上；如果预测误差很大，则说明发生了突变。该损失函数强制模型去拟合时间演化规律。$\tilde{y}_{target}^{(i)}$ 计算方式如下。

- 利用当前输入窗口 U，计算得到的统计量 μ_t, σ_t。对真实标签 u_{t+1} 执行：$\hat{u}_{t+1} = (u_{t+1} - \mu_t)/\sigma_t$。执行仿射对变换：$\tilde{y}_{\text{target}} = \hat{u}_{t+1} \odot \gamma + \beta$。

归一化的优点：1 消除协变量偏移 (Covariate Shift)：如果不进行归一化，输入是归一化后的分布 (例如 N(0,1))，而输出目标标签是原始范围 (例如 10^5 单位时)。原始的 Readout 层权重 W_{out} 必须非常大才能匹配这个量级，这会导致梯度爆炸，且极难收敛。2 几何一致性：如果不归一化目标值，我们试图用单位向

量的线性组合去拟合任意幅度的值，这破坏了“方向代表模式，模长代表强度”的几何假设。在归一化空间中，正常数据的模长相对一致，这使得 DropConnect 产生的方差具有可比性。3 鲁棒性：MCC Loss 对残差的尺度非常敏感 (由核宽 σ_{kernel} 控制)。如果目标值未归一化，很难以定义一个通用的 σ_{kernel} 来同时处理温度 (0-100) 和压力 (0-1000000)。归一化后，所有变量的残差都在同一数量级，单一的 σ_{kernel} 即可生效。

2. $\mathbb{E}_M$ (**随机性**)，即随机掩码 M。随机掩码迫使网络不能依赖某些特定的神经元组合来过拟合训练数据，而是要求 $\alpha^(i)$ 的每一个维度都必须对预测 $u(t+1)$（即 $y_{target}^{(i)}$）有贡献，且方向必须与 W_{out} 一致。这直接导致了正常数据的低方差特性。随机掩码 M 强制模型在存在权重扰动的情况下（DropConnect 函数）仍需保持预测准确，这迫使 α 必须位于 W_{out} 最鲁棒的子空间中心。$\mathbb{E}_M$ 表示对掩码分布的期望。在实际训练代码中，我们没法求精确的数学期望，而是通过蒙特卡洛采样来近似：即在每个 Batch 中，随机采样一个矩阵 M（为了区分 Batch 缩写为 M）进行计算，随着迭代次数增多，平均下来就近似于求了期望。因此，在训练时，一个 Batch 中的掩码矩阵 M 是相同的。但是，在推理时，针对一个实时多变量时序数据 $u(t)$，随机采样 K 次。

3. $\exp(-\|\cdot\|^2/2\sigma^2)$ (**高斯核**)。在时间序列异常检测的训练中，训练集往往不纯（包含历史异常）。如果使用 MSE Loss，模型会尝试拟合这些历史异常，导致 W_{out} 偏离正常流形。使用 MCC Loss 时，当遇到历史异常样本，预测误差很大，损失函数进入饱和区（梯度 ≈ 0），模型会自动拒绝学习这些异常样本。因此，即使训练集很脏，学习到的 W_{out} 仍然能指向正常数据的方向。这保证了在推理阶段，正常数据方差小，异常数据方差大（因为异常数据从未被模型接纳过）。高斯核有如下特点：

- 当预测误差很小（正常数据）时，损失函数近似于 MSE（二阶统计量），模型努力让方向 α 与 W_{out} 精确对齐。
- 当预测误差很大（训练数据中的噪声/异常数据）时，损失函数趋于常数 1，

梯度趋于 0。这意味着模型在学习 W_{out} 时，会自动切断那些被噪声污染的时间点，防止它们破坏正常数据的几何聚类。

- σ 是核宽（Kernel Size），控制对异常值的鲁棒程度。

1.3.1 参数更新公式

我们需要更新的参数集合 $\Theta = \{W_{mlp1}, b_1, W_{mlp2}, b_2, W_{out}, \gamma, \beta\}$。($W_{in}$, W_{res} 固定)。对前文的符号说明如下：其中在不影响上下文理解的情况下，省略时间符号 t，W_{mlp1}

输入状态: ($h_{res} \in \mathbb{R}^{N_{res}}$)

MLP1: ($h_{mlp} = \mathrm{ReLU}(W_1 h_{res} + b_1)$)，其中 ($W_1 \in \mathbb{R}^{N_{mid} \times N_{res}}, b_1 \in \mathbb{R}^{N_{mid}}$)

MLP2 (Embedding): ($z = W_2 h_{mlp} + b_2$)，其中 ($W_2 \in \mathbb{R}^{N_{emb} \times N_{mid}}, b_2 \in \mathbb{R}^{N_{emb}}$)

归一化: ($r = \|z\|_2$,　$\alpha = z/r = z/\|z\|_2$)

输出: ($\tilde{y}_{pred} = (W_{out} \circ M)\alpha$), 其中 $M \in \{0,1\}^{N_{out} \times N_{emb}}$

误差: ($e = \tilde{y}_{pred} - \tilde{y}_{target}$)。其中 $\tilde{y}_{\text{target}} = \mathrm{Affine}(\mathrm{Norm}(u_{t+1}))$ MCC 鲁棒权重（标量）: ($\kappa = \frac{1}{\sigma^2} \exp\left(-\frac{\|e\|^2}{2\sigma^2}\right)$)

利用链式法则进行梯度推导。

1. Loss 对预测值的梯度：

$$\delta_y = \frac{\partial \mathscr{L}}{\partial \tilde{y}_{pred}} = \kappa \cdot e$$

（这里省略了求和符号，括号对单个样本的梯度）

2. 对 W_{out} 的梯度：

$$\frac{\partial \mathscr{L}}{\partial W_{out}} = (\delta_y \cdot \alpha^T) \odot M + 2\lambda_1 W_{out}$$

更新公式：

$$W_{out} \leftarrow W_{out} - \eta \left[(\kappa e \alpha^T) \odot M + 2\lambda_1 W_{out} \right]$$

3. Loss 对 α 的梯度：

$$\delta_\alpha = \frac{\partial \mathscr{L}}{\partial \alpha} = (W_{out} \odot M)^T \delta_y = \kappa (W_{out} \odot M)^T e$$

4. Loss 对 z 的梯度（通过归一化层）：利用商规则和矩阵求导：

$$\frac{\partial \alpha}{\partial z} = \frac{1}{r}(\mathrm{I} - \alpha\alpha^T)$$

$$\delta_z = \frac{1}{r}(\delta_\alpha - (\delta_\alpha^T \alpha)\alpha), \text{check } \frac{1}{r}(\delta_\alpha - (\alpha^T \delta_\alpha)\alpha)$$

5. 对 MLP 参数的梯度

更新 W_2 和 b_2：

$$\frac{\partial L}{\partial W_2} = \delta_z h^T, \quad \frac{\partial L}{\partial b_2} = \delta_z$$

更新公式：

$$W_2 \leftarrow W_2 - \eta(\delta_z h^T + 2\lambda_2 W_2)$$

$$b_2 \leftarrow b_2 - \eta\delta_z$$

6. Loss 对 h_{res} 的梯度：

$$\delta_{h_{mlp}} = W_2^T \delta_z$$

$$\frac{\partial \mathscr{L}}{\partial W_2} = \delta_z h_{mlp}^T \quad \frac{\partial \mathscr{L}}{\partial b_2} = \delta_z$$

$$\delta_{h_{res}} = (W_1^T \delta_{h_{mlp}}) \odot \mathbb{I}([h_{mlp} > 0]) \quad \text{(ReLU 导数)}$$

7. Loss 对 MLP1 激活前的梯度：

$$\frac{\partial \mathscr{L}}{\partial W_1} = \delta_{h_{res}} h_{res}^T, \quad \frac{\partial \mathscr{L}}{\partial b_1} = \delta_{h_{res}}$$

8. 更新 W_1 和 b_1：

$$W_1 \leftarrow W_1 - \eta(\delta_{h_{res}} {h_{res}}^T + 2\lambda_2 W_1)$$

$$b_1 \leftarrow b_1 - \eta\delta_{h_{res}}$$

9. 对 RevIN 参数 γ, β 的梯度在 CESN 中，通常视储备池状态为输入的直接映射 (不更新 W_{res})。梯度通过 h_{res} 对输入 X_t 求导。

1. 对 γ 的梯度：γ 与下述无元素的乘法。

$$\frac{\partial \mathscr{L}}{\partial \gamma} = \sum_{t=1}^{L} (\delta_{\tilde{u}}[t] \odot \hat{u}[t])$$

(对 Batch 和时间维度求和)

2. 对 β 的梯度：β 是加性偏置。

$$\frac{\partial \mathscr{L}}{\partial \beta} = \sum_{t=1}^{L} \delta_{\tilde{u}}[t]$$

关于 γ, β 参数更新: 在 PyTorch 等现代深度学习框架中，梯度计算是自动完成的（Autograd）。RevIN 的参数 γ 和 β 是网络图中的叶子节点（Leaf Nodes），只要在前向传播中参与了计算，调用 loss.backward() 时框架会自动应用链式法则。不需要手动编写反向传播公式，但必须正确定义 nn.Parameter 并在前向传播中使用它。

1.4　网络训练算法

1.4.1　基于点预测的异常检测

针对 CESN 的方法，基于下一时刻预测的异常检测会更有效。ESN 的本质是利用历史记忆（Reservoir State）预测未来。预测 $u(t+1)$ 是最直接利用其记忆能力的方式。异常定义：如果 $t+1$ 时刻的真实值与基于 t 时刻记忆的预测值偏差巨大，或者模型表现出极高的不确定性，即判定 $t+1$ 时刻异常。

窗口异常：可以通过对窗口内连续的点异常分数进行平滑或投票来判定。

1.4.2　网络训练算法描述

假设我们有一个时间序列 $(u(1), u(2), \ldots, u(1000))$.

一、预热 (Warm-up) 与状态获取

注意事项：在输入储备池前，输入数据必须进行归一化；在计算 Loss 过程中，注意对真实标签（即真实值）也要进行归一化。

预热池方程: $r(t) = (1-\delta)r(t-1) + \delta \tanh(W_{\text{in}}u(t) + W_{\text{res}}r(t-1))$.

1. 初始化: 设 $r(0) = 0$.

2. Warm-up (例如前 50 步):

- $t = 1$: 输入 $u(1)$, 计算 $r(1)$.

- ...

- $t = 50$: 输入 $u(50)$, 计算 $r(50)$.

- 这 50 个状态包含了初始化的瞬态 (Transient Phase)，因此我们通常丢弃它们，不用于 MLP 的训练。

3. 状态采集 ($t = 51$ 到 999):

- $t = 51$: 输入 $u(51)$, 基于 $r(50)$ 计算出 $r(51)$。可以考虑做 50-100 步的预热。

- ...

-$t = 999$: 输入 $u(999)$, 基于 $r(998)$ 计算出 $r(999)$.

- 此时，我们得到一个状态序列集合:$S = \{r(51), \ldots, r(999)\}$.

二、构建训练集

我们将上述过程转化为监督学习样本 (Input, Target):

- 样本 1: Input $x_1 = r(51)$, Target $y_1 = u(52)$

- 样本 2: Input $x_2 = r(52)$, Target $y_2 = u(53)$

$\cdots$

- 样本 N: Input $x_N = r(999)$, Target $y_N = u(1000)$

$u(t)$ 与 $r(t)$ 的关系: $r(t)$ 是 $u(1), \ldots, u(t)$ 的连续性递归历史摘要 (History Summary)。$u(t)$ 只是当前的观测值。$r(t)$ 包含了当前值以及过去所有信息的摘要记忆。训练 MLP 的逻辑: 我们用截止到 t 时刻的历史信息 $r(t)$ 来预测 $t+1$ 时刻的观测值 $u(t+1)$。

三、训练网络 (Epoch Loop)

训练时使用 $(u_{t-i:t-1}, r(t-1))$ 作为输入数据，推理时也使用同样的窗口，所产生的效果应优于 $(u_{t-1}, r(t-1))$。窗口长度选择 10-50 之间的整数。如果异常关注的是瞬间突变，时间窗口可以小一点（如 5-10）；如果异常关注的是趋势异常，时间窗口需要大一点。

1. Batch: 随机取出一个 Batch，如 32 个状态数据。**实际训练中，可以尝试多种** Batch Size.

-输入 Batch: $R_{\text{batch}} \in \mathbb{R}^{32\times N_{\text{res}}}$

-目标 Batch: $U_{\text{target}} \in \mathbb{R}^{32\times N_{\text{out}}}$

3. Forward:

-$R_{\text{batch}} \to$ MLP $\to z \to$ Norm $\to \alpha$

-生成矩阵 M (即随机掩码), 计算 $y = (W_{\text{out}} \odot M)\alpha$

4. Backward: 计算 MCC Loss, 更新 MLP 和 W_{out}。

输入：训练序列 U，储备池参数，MLP 结构，核宽 σ，学习率 η，掩码率 p。

输出：训练好的 MLP 和 W_{out}。

1. **初始化**：随机初始化 MLP 和 W_{out}。
2. **储备池预热**：运行 ESN，生成状态序列 $R = [r(1), \ldots, r(T)]$。
3. **循环迭代** (Epochs)：
   (a) Batch Loop：随机采样 batch $(t, t+1)$。
   (b) **前向传播**：
   - $h = \text{MLP}(r(t))$
   - $z_{raw} = \text{Linear}(h)$
   - $r = \|z_{raw}\|, \quad \alpha = z_{raw}/r$
   - 生成掩码 $M \sim \text{Bernoulli}(1-p)$
   - $\hat{y} = (W_{out} \odot M)\alpha$

   (c) **计算误差与权重**：

- $e = \hat{y} - u(t+1)$
- $w_{robust} = \exp(-\|e\|^2/2\sigma^2)$

(d) **反向传播与更新：**

- 计算梯度 $\nabla\Theta$，其中 W_{out} 的梯度需乘以 w_{robust} 和掩码 M。
- $\Theta \leftarrow \Theta - \eta\nabla\Theta$

5. **统计正常数据的特征**

- 在验证集上计算平均模长 $\bar{r}_{\text{train}}$ 和基准方差 $\bar{v}_{\text{train}}$。

1.5 实时推理算法

1.5.1 输入

输入应该是时间窗口中的多个多变量数据 (组阵), 即 $U_t \in \mathbb{R}^{L\times N_{var}}$, 而不是单个向量。原因如下：1 RevIN 的核心是对当前实例进行归一化。它需要计算当前输入的均值 μ 和标准差 σ。如果输入仅是 t 时刻的一个向量 $u_t \in \mathbb{R}^{N_{var}}$, 我们只能计算单个向量的统计量 (没有意义，因为不同传感器物理量纲不同)，或者无法计算（单个时间点的时域方差为 0）)。为了去除非平稳性 (Non-stationarity), 我们需要拥有一个局部上下窗口 (Look-back Window), 在该窗口内计算 μ_t, σ_t 才有统计学意义, 才能消除局部趋势, 保留形态特征。

2. ESN 的状态初始化 (Warm-up):

1.5.2 异常得分公式

多变量时间序列（MTS）异常检测:

- 目标空间的动态性：MTS 异常检测应用中，目标是 $t+1$ 时刻的数值 y_{t+1}，注意到这里 y_{t+1} 是一个向量，即多变量的预测值。这是一个动态变化的目

标。即使 α 在嵌入空间很稳定，如果它映射出的 $\hat{y}$ 偏离了真实的 y_{t+1}，这也是异常（预测误差）。

- 变量间的相关性 (Correlations): MTS 的核心难点在于变量间的依赖关系 (Inter-variable dependency)。嵌入向量 α 是一个压缩的共享表示 (Shared Representation)。读出矩阵 W_{out}，承担了解码变量间复杂关系的任务。如果我们只看 α，我们忽略了 W_{out} 中蕴含的“哪些变量组合是合法的”这一信息。
- 方法的物理意义: 当 DropConnect 作用于 W_{out} 时，它在输出空间 $\hat{y}$ 上产生的方差 $\mathrm{Var}(\hat{y})$。实际上是嵌入空间几何不确定性 $\mathrm{Var}(\alpha)$ 经过 W_{out} 加权后的投影。如果某些变量很难预测（噪声大），W_{out} 对应的权重可能较小，自动抑制了这部分的方差；反之亦然。直接使用 $\hat{y}$ 的方差能更好地反映“针对当前任务”的不确定性。

对于基于 CESN 的多变量时间序列异常检测，应使用基于预测值 $\hat{y}$ 的统计量，并结合模长信息。Score 的计算考虑在 RevIN 归一化后的空间进行。异常分数公式如下：（三种方式，可能后两种方式比较有效。Intuitive 上，参数是否容易搜索？多变量是多少维的？以及三项归一化引发的问题。哪种参数跟容易所搜？或确定，或取决变量是多少维的？）

$$S_t = \sum_{i=1}^{N_{\mathrm{var}}} \left(\underbrace{\left(\tilde{y}_{\mathrm{target},i} - \bar{\tilde{y}}_{\mathrm{pred},i}\right)^2}_{\text{Norm. Error}} + \varphi_1 \cdot \underbrace{\mathrm{Var}\left(\tilde{y}_{\mathrm{MC},i}\right)}_{\text{Norm. Uncertainty}} \right) + \varphi_2 \cdot \underbrace{\left(1 - \frac{r_t}{\mu_r}\right)^2}_{\text{Geometry}} \tag{1.3}$$

$$S_t = \frac{1}{N_{var}} \sum_{i=1}^{N_{var}} (\tilde{y}_{\mathrm{target},i} - \bar{\tilde{y}}_{\mathrm{pred},i})^2 + \varphi_1 \cdot \frac{1}{N_{var}} \sum_{i=1}^{N_{var}} \hat{\sigma}^2_{\mathrm{MC},i} + \varphi_2 \left(1 - \frac{r_t}{\mu_r}\right)^2 \tag{1.4}$$

$$S_t = \underbrace{\frac{1}{N_{var}} \sum_{i=1}^{N_{var}} \frac{(\tilde{y}_{target,i} - \bar{\tilde{y}}_{pred,i})^2}{\hat{\sigma}^2_{hist,i}}}_{\text{Term 1: Weighted Accuracy}} + \varphi_1 \cdot \underbrace{\frac{1}{N_{var}} \sum_{i=1}^{N_{var}} \hat{\sigma}^2_{MC,i}}_{\text{Term 2: Mean Uncertainty}} + \varphi_2 \cdot \underbrace{\left(1 - \frac{r_t}{\mu_r}\right)^2}_{\text{Term 3: Geometry}} \tag{1.5}$$

1. 所有计算均在 RevIN 归一化空间进行。
2. $\bar{\tilde{y}}_{pred,i}$ 是第 i 个变量在 K 次 MC 采样中的均值。$\hat{\sigma}^2_{MC,i} = \frac{1}{K-1}\sum_{k=1}^{K}(\tilde{y}_{k,i} - \bar{\tilde{y}}_{pred,i})^2$ 是第 i 个变量的预测方差。
3. $r_t = \|z_t\|_2$ 是嵌入模长。
4. 包含 $\frac{1}{N_{var}}$ 的分数: 它将 MSE 和 Variance 转化为“每个传感器的平均异常程度”。原则上：不会降低检测灵敏度。因为异常检测是看相对变化。虽然绝对数值变小了，但背景噪声的数值也按同样比例变小了，信噪比 (SNR) 保持不变。
5. 第三个公式中为尺度不变（Scale-Invariant）且维度不变（Dimension-Invariant）的分数设计。引入 $\hat{\sigma}^2_{hist,i}$：虽然 RevIN 已经归一化了输入，但不同变量在预测时的残差分布可能不同（有些变量噪声本来就大，有些很平滑）。除以该变量历史残差方差 $\hat{\sigma}^2_{hist,i}$，实现了逐变量的信噪比加权。比简单的 MSE 更符合直观数据利用中“Log-Likelihood”的思想。平均化 (1/N)：保证了 Term 1, Term 2 与 Term 3 在数量级上接近（都是 O(1)），使得 φ 的调节更加鲁棒。

第一项 Accuracy (MSE) **或** Norm. Prediction Error 。在归一化空间计算 MSE。这保证了尺度不变性（Scale Invariance）。微小变量的剧烈波动（如 $0.1 \to 0.8$）贡献的 Loss 与大变量的剧烈波动（如 $1000 \to 8000$）相当。预测误差是时序异常检测的基础。如果预测偏离真实值，可视为异常。**物理意义：**预测偏差。衡量当前时刻的数值是否符合历史规律。**针对的异常类型：** Point Anomalies (点异常)。突变、大幅偏离。**为什么用于** MTS：这是最直观的异常。如果真实值突然偏离了基于历史信息的预测值，就明显发生了违反物理规律的事件。

第二项 Uncertainty (Variance) **或稳定性**（Norm. Uncertainty）。**物理意义：**认知不确定性。衡量模型对当前输入模式的“陌生程度”。**针对的异常类型：** Contextual Anomalies (上下文异常)、未知模式、分布偏移 (Concept Drift)。**为什么用于** MTS：某些异常可能数值不大（MSE 小），但组合模式从未见过。此

时 DropConnect 会导致不同子网络预测不一致，方差剧增。这是检测“隐性异常”的关键。使用归一化输出 $\tilde{y}$ 的方差。第二项是输出空间的映射。这衡量了模型对当前模式的不确定性，排除了数值大小本身带来的干扰。K 次蒙特卡洛 DropConnect 产生 K 个 $\tilde{y}_{pred}$。如果模型对当前输入感到陌生，预测值会发散，方差剧增。用于度量不确定程度。在多变量时间序列异常检测中，数值方差之和（Sum of Scalar Variances）（等价于协方差矩阵的迹 Trace。我们的问题是检测 N_{var} 个传感器中是否存在任意一个出现异常。DropConnect 在 W_{out} 产生的随机性，经过线性变换投影到了输出空间 $\tilde{y} \in \mathbb{R}^{N_{\text{var}}}$。我们关心的是输出空间中每个变量的不确定性。数学等价性：对于多变量向量 y，其总变差 (Total Variation) 定义为协方差矩阵的迹 $\text{Tr}(\text{Cov}(y))$。而 $\text{Tr}(\text{Cov}(y)) = \sum_{i=1}^{N_{\text{var}}} \text{Var}(y_i)$。计算框架：计算每个维度的方差再求和，复杂度是 $O(K \cdot N_{\text{var}})$；而计算完整的协方差矩阵再求迹，复杂度更高且没必要。

第三项 Geometry (Norm) **特征激活强度或分布偏移**（Geometry）。**物理意义：**衡量输入数据在特征空间中的“能量”或“离群度”。**针对的异常类型：** 异常 Inputs、传感器死锁（模长趋零）、传感器饱和（模长极大）。**为什么用于** MTS：随机嵌入论文指出正常数据模长较大且集中。异常往往导致模长显著变小（特征提取器未激活）或显著变大（未见过的强信号）。如果模长显著偏离，说明输入数据在特征提取阶段就被 ESN 判定为“异类”，即使强行预测也不可信。第三项无需归一化，因为模长 r_t 本身与变量维度无关，它是嵌入空间的属性，因此保持原样。

这三项构成了一个互补的检测系统。Accuracy 负责“显性错误”，Uncertainty 负责“隐性陌生”，Geometry 负责“特征层面的离群”。这种异常分数设计同时考虑了性能（Accuracy）、置信度（Confidence）和分布特征（Distribution）。仅依靠预测误差会漏掉“预测正确但模型极度不确定”的场景；单纯依靠 α 方差会忽略“模型很确定但预测完全错误”的场景（例如突变点）。

1.5.3 异常分数的动态归一化方法 (Dynamic Normalization)：

关于异常分数设计，考虑下述两种方式，通过实验考察哪种效果更优。为了符号清晰以及便于理解，我们引入简单的符号分别对应异常分数的三个部分：

$$S_t = E_t + \varphi_1 V_t + \varphi_2 D_t$$

考虑到这三项的物理量纲不同，直接相加的效果可能不理想。具体是否需动态归一化以及是否使用搜索的方式确定参数，需要看实验效果。

1. Grid Search (**搜索方式：如果有验证集**)：

在包含少量异常的验证集上，最大化 F1-Score 搜索 φ_i。通常 φ_1 (不确定性) 权重较高，因为对于异常模式的数据，Error 可能不大（如果模型仅仅是输出均值），但 Variance 会比较大。

2. **动态归一化策略** (Dynamic Scaling)：

建议第二种方法归一化策略？取决于这三项分数的量纲 尽管在 RevIN 空间计算，解决了变量间的量纲问题，但 Score 三个分项本身的可能物理量级差异大主要原因包括：量级的不平衡和基准线的漂移。

1. 量级的不平衡分析如下：

- 第一项：在异常发生时，归一化后的残差平方可能达到 $10-100$ 的量级（例如偏离 3-10 个标准差）。
- 第二项：由于 DropConnect 是微扰权重，预测值的方差通常很小，可能在 $0.01-0.1$ 的量级。
- 第三项：模长偏差通常是一个比率的平方，量级在 $0-1$ 之间。

2. 基准线的漂移 (Baseline Drift)：分析如下：

随着时间推移，模型的预测性能可能因环境变化而缓慢下降（MSE 基准线抬升）。如果不进行动态调整，Score 会整体漂移，导致误触发固定阈值或 POT 阈值适应迟滞。

动态归一化策略不设定固定的 φ_i 值，而是将每一项除以其在验证集（或历史窗口）上的均值或标准差。考虑采用滑动平均标准化 (Moving Average Normalization)。单变量计算出 S_t 后，考虑到三个项的波动幅度不同，不能直接用于阈值。需要使用局部 Z-Score 标准化 (Local Z-Score Normalization)。

计算步骤：对每一项 $k \in \{1,2,3\}$(分别代表 Accuracy, Uncertainty, Geometry)：

1. 维护滑动窗口统计量：维护一个长度为 W 的历史窗口，计算该分项的均值 $\mu_{k,t}$ 和标准差 $\sigma_{k,t}$。注意到前面也提到了窗口，虽然都用了符号 W，但表示不同的窗口大小。在这里，均值和方差与前文虽然也使用了相同的符号，但是计算时，是针对不同输入数据的均值方差。

2. 计算标准化子分数：

$$Z_{k,t} = \frac{|\text{Term}_{k,t} - \mu_{k,t}|}{\sigma_{k,t} + \epsilon}$$

注意：如果是突发异常，$\text{Term}_{k,t}$ 会远大于均值，导致 $Z_{k,t}$ 很大 (例如 >3)。

3. 最终融合：

$$S_{\text{final},t} = Z_{1,t} + Z_{2,t} + Z_{3,t}$$

这种方法的优势：消除了量纲困扰：无论我们是否除以了 N_{Var}，Z-Score 变换都会将分数映射到“偏离正常多少个标准差”的无量纲空间。除以 N_{Var} 的作用：它保证了在进入 Z-Score 计算前，数据的数值稳定性，避免浮点数溢出或精度损失。融合了所有异常视角：无论异常表现为预测错误、不确定性增加还是模长改变，只要其中一项的 Z-Score 飙升，总分就会飙升。

1.5.4　动态（或自适应的）异常分数阈值设计策略

对于流式数据，如果数据分布发生漂移，或者出现在此前未见的“正常”波动，静态阈值会导致大量误报或漏报。引入 POT (Peaks-Over-Threshold) 方法，来动态设定阈值。其主要思想：不需要假设异常分数服从高斯分布（通常情况不服从该分布）。我们只关注分布的尾部（Tail）。极值理论 (Extreme Value Theory,

EVT) 证明了无论原分布如何，超过某一高阈值的尾部数据都服从广义帕累托分布 (GPD)。自适应性：阈值随数据流动态调整。异常分数调整步骤如下：

1. **收集分数：** 维护一个滑动窗口内的历史异常分数集 $S_{\text{history}} = \{S_{t-W}, \ldots, S_t\}$。注意，每个时刻的多变量数据都有一个异常分数，不是只有异常数据才有异常分数。我们根据这个分数判定是否发生异常。

2. **初始阈值选定：** 选择 S_{history} 的一个高分位数（如 98%）作为基准阈值 th_{init}.

3. **拟合 GPD：** 筛选出所有超过 th_{init} 的分数（Peaks），拟合 GPD 参数 $(\hat{\xi}, \hat{\kappa})$.

$$F_{th_{\text{init}}}(x) = P(X - th_{\text{init}} > x \mid X > th_{\text{init}}) \sim \left(1 + \frac{\hat{\kappa} x}{\hat{\xi}}\right)^{-\frac{1}{\hat{\kappa}}}.$$

4. **计算最终阈值 z_q：** 给定极小的风险概率 q（如 10^{-4}），阈值计算公式为

$$z_q = th_{\text{init}} + \frac{\hat{\xi}}{\hat{\kappa}}\left(\left(\frac{q \cdot |S_{\text{history}}|}{N_{\text{peaks}}}\right)^{-\hat{\kappa}} - 1\right).$$

th_{init} 是区分“正常波动（Normal Noise）”和“极值尾部（Extreme Tail）”的分界线。EVT 理论（Pickands 定理）仅在超过这个阈值的数据上成立。th_{init} 选得太低：正常数据混入尾部，破坏 GPD 分布假设，导致最终阈值 z_q 估计不准。th_{init} 选得太高：超过阈值的样本数 $|N_{\text{peaks}}|$ 太少，导致参数估计方差极大。通常 98% 是经验上的最佳平衡点。$|S_{\text{history}}|$ 表示历史异常分数集的大小，参考上述符号为 $|w+1|$。$|S_{\text{history}}|$ 越小，阈值适应得越快（适应概念漂移），但统计稳定性越差；$|S_{\text{history}}|$ 越大，阈值越稳定，但对突发变化的反应越迟钝。

$N_{peaks} = \sum_{S_i \in H} \mathbb{I}(S_i > th_{\text{init}})$。$|N_{\text{peaks}}|$ 表示所有超过 th_{init} 的分数的个数，即有多少个时刻的数据，其对应的异常分数高于 th_{init}。在历史窗口中，实际超过初始阈值 th_{init} 的样本数量（即“峰值”的数量）。N_{peaks} 是 GDP 多数据拟合的实际样本量。在公式中，比率 $N_{peaks}/|S_{\text{history}}|$ 代表了先验的尾部概率。公式中的项 $\frac{q}{N_{peaks}/|S_{\text{history}}|}$ 本质上是在计算：我们需要从当前的尾部概率 $N_{peaks}/|S_{\text{history}}|$ 推演到更极端的风险概率 q 时，需要向外推 (Extrapolate) 多远。

CESN 提供了高质量的、包含不确定性的分数 $S(t)$，而 POT 提供了统计学上严谨的、自适应的阈值 z_q。

为了计算异常分数阈值，我们需要给出 GDP 分布的参数估计，本文采用矩估计方法。**基于矩估计** (MOM) **的** GPD **参数估计优点**: 矩估计具有闭式解（Closed-form solution）。计算公式仅涉及均值和方差，复杂度为 $O(1)$（在维护流式均值方差的前提下）。数值比较稳定。缺点：精度略低于 MLE，但在异常检测的阈值设定场景下，这种精度损失完全可以接受，换取的是高吞吐量。结论：对于实时（Real-time）或流式（Streaming）异常检测，选择 MOM。

1. 数据准备：取历史异常分数集合 S。选取一个阈值 th（例如 S 的 98% 或 95% 分位数）。

2. 计算超额量 (Exceedances)：找出所有 $S_t > th$ 的样本，计算 $f_i = S_t - th$。构成样本 $\{f_1, \ldots, f_k\}$。

3. 拟合参数：估算 GPD 的形状参数 ξ 和尺度参数 κ。

为了低延迟，我们使用矩估计法 (Method of Moments, MOM)，闭式解。计算超过量的均值 $\bar{f}$ 和方差 d^2。估算公式如下：

$$\kappa = \frac{1}{2}\left(1 - \frac{\bar{f}^2}{d^2}\right), \qquad \xi = \frac{1}{2}\bar{f}\left(\frac{\bar{f}^2}{d^2} + 1\right)$$

形状参数 ξ 和尺度参数 κ 是需要实时估计的，随滑动窗口更新。计算复杂度极低，是 $O(1)$ 是常数级别的。

动态阈值判定 (POT) 方法:

- 使用历史异常分数集合 H 执行 GPD, 计算动态阈值 z_q。
- If $S_t > z_q$:
  - Alarm: 异常!
  - Action: 不更新 H (避免污染阈值模型型)。
- Else:

- Normal: 正常。
- Action: 更新 $H \leftarrow H \cup \{S_t\}$, 移除最旧数据。

1.5.5 异常检测算法

输入：实时数据 $u(t)$，模型参数，蒙特卡罗（MC）次数 K，阈值 τ。

1. 更新储备池得到 $r(t)$。
2. 通过 MLP 得到 z_{raw}，计算

$$r(t) = \|z_{\text{raw}}\|, \quad \alpha = \frac{z_{\text{raw}}}{r(t)}.$$

3. 蒙特卡洛方差估计：
   - 生成 K 个掩码 $M_1, \ldots, M_K$。
   - 计算 K 个预测：

$$y_k = (W_{\text{out}} \odot M_k)\alpha.$$

   - 计算预测均值：

$$\bar{y} = \frac{1}{K}\sum_{k=1}^{K} y_k.$$

   - 计算预测方差：

$$V(t) = \frac{1}{K}\sum_{k=1}^{K} \|y_k - \bar{y}\|^2.$$

4. 计算异常得分：

$$S(t) = V(t) + \gamma \frac{|\bar{r}_{\text{train}} - r(t)|}{r(t)}$$

   注：结合方差（不确定性）和模长（特征偏离度）。

5. 判定：
   - 若 $S(t) > \tau$，则判定为异常。

1.5.6 算法解释

1. 训练样本：打散的（Shuffled）状态 -目标对 $\{(r(t), u(t+1))\}_{batch}$ 。打散效果更好，因为它打破了 SGD 训练中的时间相关性，使梯度估计更稳定。网络最终的输出 $\hat{y}(t)$ 是下一时刻时间序列数据的预测值（Prediction in Observation Space），即预测 $u(t+1)$。

2. Warm-Up 阶段：由 ESN 的状态公式可知，$r(t)$ 依赖于 t 时刻之前的所有历史。程序刚启动时，$t=0$，我们通常设 $r(0)=0$。但这并不是真实的系统状态。如果不处理，前几十个时刻的状态 $r(1),\ldots,r(50)$ 都是错误的（受初始值 $\{0\}$ 的影响，称为 transient phase）。因此，把数据的前 W 个点（例如 50 个点）喂给网络，更新 r，但是不记录 Loss，也不进行反向传播。这样可以让储备池遗忘掉人为设置的初始零状态，进入真正的动态轨迹。同时需要注意的是，在推理阶段，也需要进行预热，然后再进行推理算法。也就是说，必须保证每次计算 $r(t)$ 时，你的储备池已经热身过了，不能突然从中间断开的地方开始算，否则状态是错的。

1.6 增量网络更新方法 (Incremental Learning)

1.7 适用的数据集及其评价指标

1.7.1 适用的数据

高噪声、非平稳的真实数据集：

- SMD (Server Machine Dataset) & MSL/SMAP (NASA)：这些数据集包含复杂的时空依赖，且有很多间歇性噪声。
- SWaT (Secure Water Treatment)：包含物理过程的多变量依赖。
- 不适用：简单的线性数据集或极其干净的模拟数据。

DropConnect + ESN 擅长处理小样本和高噪声。深度 Transformer 模型在数据量少或噪声极大时容易过拟合，而 CESN 的随机性和几何约束起到了强正则化

作用。

多变量依赖：MTS 的异常往往不是单变量值的跳变，而是**变量间关系的破坏**（例如：阀门打开但流量没变）。CESN 的全连接 Readout 层捕捉这种 Correlation。

1.7.2 优势指标

我们的方法在以下指标上会有显著优势：

- F1-Score (尤其是 Precision)：传统方法（如单纯 MSE）容易因噪声产生大量误报（FP）。CESN 利用方差（不确定性）和 MCC Loss 过滤掉噪声引起的误差。如果误差大但方差也大，说明模型只是不确定，而非绝对异常；或者通过 MCC 忽略了训练中的噪声。这会显著降低 FP，提高 Precision。
- Detection Delay (延迟)：ESN 的递归特性对时间突变非常敏感，加上 EVT 动态阈值的快速响应，检测延迟通常低于基于滑动窗口的 Transformer。
- 鲁棒性 (Robustness to Contamination)：在训练集包含异常（Training Data Contamination）的情况下，MCC Loss 保证了模型不会拟合异常，这是大多数 MSE-based AutoEncoder 做不到的。

1.8 程序调试也许可用方法或遇到问题

Loss 函数调参注意 (该段叙述也许不正确，看程序运行)：通常 λ_1 (针对 W_{out}) 可以稍大，以满足几何约束; 而 λ_2 (针对 MLP) 通常较小 (如 $1e-4$ 或 $1e-5$), 仅用于防止过拟合，不要阻碍特征提取。Loss 曲线下降可能会变缓，但验证集上的性能 (尤其是异常检测的 AUC) 通常会更高且更稳定。

(该段叙述也许不正确，看程序运行)：1. 梯度清零: 每次迭代前必须 optimizer.zero grad()。2. ε: 在做归一化除法时，务必加一个极小值 $1e-8$，防止 norm 为 0 导致 NaN。3. Loss 函数中 σ 选择: σ 太大，Loss 退化成 MSE; σ 太

小，梯度消失，模型学不动。通常需要根据数据的方差进行调整 (例如取数据标准差的一半？)。

两层或三层 MLP 应该可以?

1. 初始化优化问题：ESN 的性能高度依赖于 W_res 的随机初始化、谱半径（Spectral Radius）和稀疏度。如果运气不好，初始化的储备池可能不具备良好的“回声特性”（Echo Property），导致无法区分正常和异常。需要进行超参数搜索或使用特定的拓扑结构（如 SCR - Simple Cycle Reservoir）。

2. 数据集问题：ESN 本质上是一个离散的非线性动力系统，因此对于具有明确物理规律的数据效果最好。Mackey-Glass (MG) 时间序列 (合成)，Lorenz Attractor (合成)，ECG 心电图数据 (如 MIT-BIH Arrhythmia Database), NASA Telemetry Data (SMAP / MSL),

1.8.1 评估指标

这里仅给出建议，可能不准确。建议进行广泛调研，Benchmark 那篇论文会给出较多启发。你还需分析后，进行调研，实验考察最终结果。

1. AUROC **基准指标**: AUROC (Area Under Receiver Operating Characteristic): 异常检测通常不预设阈值。AUROC 衡量的是" 无论阈值取多少，你的分数把异常排在正常前面的能力有多强"。这是学术界认可的指标。

2. AUPR (Area Under Precision-Recall Curve): 原因: 如果数据集极其不平衡 (异常只有 0.1%)，AUROC 可能会虚高，AUPR 更能反映真实能力.

3. F1 Score

画图或计算以下指标:

- 正常数据 (ID) vs. 异常数据 (AD) Variance Gap (方差间隙):
  - 计算测试集中所有 ID 样本的几何方差均值 μ_{ID}。
  - 计算测试数据集所有异常数据样本的几何方差均值 μ_{AD}。

  - 指标：使用 $|\mu_{ID} - \mu_{AD}|$ 或 $\frac{|\mu_{ID}-\mu_{AD}|}{\mu_{AD}}$ 越大越好（自然假设 $\mu_{ID} > \mu_{AD}$）。

- Score Distribution Histogram (分数分布直方图):
  - 画出 ID 样本的方差分布（例如蓝色）和 AD 样本的方差分布（例如红色）。
  - 好模型：两个分布像两座山峰一样离得越远越好 (重叠面积越小越好)。

1.9 参数调试问题

精确度高但是召回率很低，这是在异常检测（Anomaly Detection, AD）实验中经常遇到的问题：高精确度 + 低召回率。这可能意味着：我们的模型非常保守或谨慎。它只要报警，基本都是真的异常（Precision 高）。但是它漏掉了很多真实的异常（Recall 低，漏报率高）。原因：异常阈值（Threshold）切得太高了，或者异常分数的动态范围（Dynamic Range）被压缩了。

1.9.1 1. 第一优先级：EVT 参数调节 (Inference Stage)

这是最直接、不需要重新训练模型就能解决问题的手段。问题大概率出在 POT (Peaks-Over-Threshold) 的参数 q 上。阈值公式为：

$$z_q = th_{\text{init}} + \frac{\hat{\xi}}{\hat{\kappa}}\left(\left(\frac{q \cdot |S_{\text{history}}|}{N_{\text{peaks}}}\right)^{-\hat{\kappa}} - 1\right).$$

其中 q 是预设的风险概率（Risk Probability）。Recall 低，说明 z_q 高了。可能设定的 q 太小了（例如 10^{-4} 或 10^{-5}）。这使得模型只敢把那些十分离谱的点判为异常，稍微温和一点的异常都被放过了。

可能得解决方法：增大 q 值：尝试范围 **：如果现在是 10^{-4}，尝试增大到 10^{-3}，甚至 3×10^{-3}。物理含义：允许更多的样本进入异常候选区。这会直接降低阈值 z_q，从而捕获更多异常点，显著提升 Recall。代价：Precision 可能会轻微下降（误报变多），但 F1-score 通常会上升，直到达到平衡点。

1.9.2 2. 第二优先级：Score 的动态归一化策略 (Score Normalization)

Score 的动态归一化策略里的参数设置也比较重要。$S_{\text{final},t} = Z_{1,t} + Z_{2,t} + Z_{3,t}$，其中

$$Z_{k,t} = \frac{|\text{Term}_{k,t} - \mu_{k,t}|}{\sigma_{k,t} + \epsilon}$$

如果滑动窗口的统计量（μ_t, σ_t）更新得太快，异常本身会被吸入到背景分布中，导致 $S_{final,t}$ 变小，无法超过阈值。

解决方法：1. 增大滑动窗口长度：确保统计量的计算基于足够长的历史数据（例如从 500 增加到 2000）。2. 调整平滑系数 α：如果你用 ($\mu_t = \alpha S_t + (1-\alpha)\mu_{t-1}$)，减小 α（例如从 0.01 改为 0.001）。让历史基准线更稳定，不要因为异常出现就迅速抬高基准线，从而掩盖了后续的异常。

1.9.3 3. 第三优先级：评估标准 (Point Adjustment, PA)

评估标准也是比较关键的问题。在时间序列异常检测的顶会论文（如 OmniAnomaly, USAD, TranAD 等）中，绝大多数都使用了 Point Adjustment (PA) 策略。

PA 的定义：如果一个连续的异常区间（例如 $t = 100$ 到 $t = 110$ 是真实异常）中，模型只要检测到了其中任意一个点（例如检测到了 $t = 105$），就认为整个区间都被正确检测到了。如果没有使用 PA，而是使用 Point-wise（逐点）计算 Recall，那么 Recall 天然会非常低。因为模型往往只能检测到异常发生的瞬间（头部）或最剧烈的时刻（峰值），而漏掉了异常的起始和结束边缘。

解决方案：是否使用了 Point Adjustment。论文中应使用 PA，否则无法与 Baseline 公平对比（Baseline 的高 Recall 90% 都是 PA 带来的）。

1.9.4 4. 第四优先级：Score 的平滑处理 (Smoothing)

CESN 的输出（特别是 DropConnect 的方差项）可能存在高频抖动。真实异常通常是一个持续的过程（Continuous Segment）(这里，是否为持续过程取决于数据集的特点)。模型的 Score 可能忽高忽低（Spikes）。如果没有平滑，阈值可能会切断异常的腰部，导致 Recall 低（只抓住了尖峰）。

解决方案：在计算最终 Score 后，进入 POT 阈值之前，先加一个滑动平均滤波：$S_{smooth,t} = \text{MovingAverage}(S_t, \text{window} = k)$ 建议 $k = 5$ 或 $k = 10$。效果：会填补异常分数中的短期低谷，把断断续续的报警连成一片，显著提升 VUS 和 Recall。

1.9.5 第五优先级：MCC Loss 的核宽 σ (Training Stage)

如果上述推理阶段的方法都无效，可能需要重新训练。

MCC Loss: $\mathscr{L} = 1 - \exp(-\|e\|^2/2\sigma^2)$。问题：如果训练时的核宽 σ 设置得过大。MCC Loss 退化为 MSE Loss。模型会过度拟合训练数据中的隐性异常或噪声。在测试时，遇到异常数据，模型也能拟合得比较好（泛化能力太强，错把异常当正常），导致预测误差（Term 1）不够大，Score 不高，导致漏报（Low Recall）。

解决方案：减小训练时的 σ。让模型对误差更敏感，强迫模型只学习最主要、最干净的规律。对于稍微偏离规律的数据，模型学不会，这样在测试时，异常数据的预测误差就会很大，从而容易被检测出来。

1.9.6 6. 第六优先级：数据不平衡与采样

虽然是无监督，但如果训练集中混入了过多的异常（污染率高），且你用的 σ 没有足够小以过滤它们，模型学会了异常的样子。

解决方案：如果训练集污染严重，确保 MCC 的 σ 足够小（利用 MCC 的鲁棒性拒绝学习异常）。或者在训练前做简单的预清洗（Pre-cleaning），剔除明显的离群点。

1.9.7 Score 的超参数 φ_i

对测试/推理效果的影响 (Impact on Inference)（基于 RevIN 空间）的分数：$S_t = \text{Error}_t + \varphi_1 \cdot \text{Uncertainty}_t + \varphi_2 \cdot \text{Geometry}_t$

φ_1 (不确定性/方差权重) 的影响：它控制模型对未知模式 (Unknown Patterns) 的敏感度。如果设置过大：模型会变得非常敏感。只要输入数据稍微有一点没见过的噪声或波动，DropConnect 产生的方差就会波动，导致 S_t 飙升。Recall 上升，Precision 下降。你会抓到所有的异常，但也会把正常的噪声突变当成异常（误报率高）。如果设置过小：模型退化为纯预测模型。它只能检测到数值偏差大的异常。Recall 下降。对于那些数值看起来正常，但内部逻辑错乱的上下文异常 (Contextual Anomalies)（例如：本该正相关变成了负相关），预测均值可能偏差不大，但方差会很大。如果忽略 φ_1，这部分异常就会被漏掉。

φ_2 (几何/模长权重) 的影响：它控制模型对特征强度的敏感度。如果 φ_2 设置过大：模型对信号的能量变化极度敏感。擅长检测传感器死锁 (Dead Sensors)（模长趋零）或极端冲击 (Extreme Shocks)（模长爆炸）。但也容易受 RevIN 归一化不稳定的影响（如果滑动窗口内的统计量突变），导致误报。如果 φ_2 设置过小：模型忽略特征空间的分布约束。漏报异常数据。有些异常输入经过 ESN 映射后，可能刚好落在 W_{out} 的盲区，导致预测值不大（Error 小）、方差也不大（Uncertainty 小），唯独模长极小（特征未激活）。如果没有 φ_2，这种隐身的异常就检测不到。

在实验中平衡 λ_1 和 λ_2。如果高 Precision 低 Recall，这说明 S_t 对某些真实的异常特征不敏感。

动态 Z-Score 归一化？如前文所述，不要手动调 λ_1, λ_2 的具体数值，而是对三项分别做 Z-Score：$S_{final} = Z(\text{Error}) + Z(\text{Uncertainty}) + Z(\text{Geometry})$

可以基于验证集的网格搜索 (Grid Search)：固定 $\varphi_1 = 0, \varphi_2 = 0$：只看 Error，计算 Recall。逐步增加 φ_1：直到发现 Recall 显著提升，同时 Precision 没有大幅下降。这通常能找回那些数值偏差不大但模式异常的样本。逐步增加 φ_2：针对那

些信号丢失或传感器故障的样本。

低 Recall：如果模型漏报多，很可能是因为很多异常表现为高不确定性而非高误差。尝试增大 φ_1，让模型对陌生模式更敏感。可以使用 Z-Score 动态归一化来替代手动设定这两个超参数，这能让模型自动平衡三项的权重，通常能直接获得最佳的 Precision-Recall 平衡。

1.9.8 Loss 超参数

1. λ_1（Readout 层正则化）的影响：决定不确定性的有效性。直接影响：异常分数中的第二项（Uncertainty/Variance）。DropConnect 网络的训练等价于贝叶斯推断，其中 L_2 正则项 $\lambda_1\|W_{out}\|^2$ 对应于权重的先验分布（Prior Distribution）$p(W) \sim \mathcal{N}(0, \tau^{-1}I)$。

如果 λ_1 过小（接近 0）。物理含义：相当于假设权重的先验方差无穷大（Uniform Prior）。W_{out} 的模长会不受控制地变大，以拟合训练数据中的微小噪声。对 Score 的影响：1. 方差爆炸：DropConnect 的输出方差 Var $\approx p(1-p)\|W_{out}\|^2\|\alpha\|^2$。如果 $\|W_{out}\|$ 很大，计算出的 Uncertainty（分数第二项）会变得非常大且不稳定。2. 掩盖真实异常：正常数据的方差本身就很大（因为权重过大），导致异常数据的方差被淹没，信噪比（SNR）降低。结果: Low Precision。

如果 λ_1 过大。物理含义：假设权重的先验极其集中于 0（Strong Prior）。后果：W_{out} 被强力压缩接近于 0。模型倾向于输出 0（即 RevIN 的均值）。对 Score 的影响：1. 方差消失：Uncertainty 项几乎为 0，模型失去通过不确定性检测未知模式的能力。2. 退化为 MSE：Score 退化为仅依赖预测误差。对于那些数值正常但逻辑错误的隐性异常，模型无法检测。结果: Low Recall。

结论：λ_1 必须适中（通常在 10^{-3} 到 10^{-5} 量级？）。它是校准（Calibrate）不确定性分数的关键旋钮。

2. λ_2（MLP 层正则化）的影响：决定几何特征的质量。直接影响：异常分数中的第三项（Geometry/Norm）以及第一项（Accuracy）。

MLP 负责将储备池的混沌状态 r 映射到超球面嵌入空间。如果 λ_2 过小。物理含义：允许 MLP 学习极其复杂的非线性映射。后果：MLP 会“死记硬背”训练集中的每一个状态，甚至将噪声映射到超球面上特定的点。对 Score 的影响：1. 流形破碎：正常数据的嵌入在超球面上不再紧密聚类（Concentration 减弱）。2. 几何分数失效：由于 MLP 过拟合，它可能将异常输入也强行映射到正常簇附近（模长正常），导致 Score 第三项失效。结果: Low Recall (对异常数据不敏感)。

如果 λ_2 过大。物理含义：迫使 MLP 变为简单的线性映射或常数映射。后果：所有输入都被映射到同一个方向，或者模长都很小。对 Score 的影响。1. 特征坍塌 (Feature Collapse)：嵌入向量失去区分度，模型无法区分不同的时间模式。2. 预测误差增大：由于特征表达能力不足，Score 第一项（Error）在正常数据上也很大，导致高误报。Result: Low Precision。

λ_2 保证了嵌入空间的平滑性 (Smoothness)。它确保了相似的储备池状态映射到相似的嵌入向量，这是几何异常检测成立的前提。

3. 与 σ（核宽）的对比与协同

σ (Kernel Width)：控制模型对 Label Noise（标签噪声/离群点）的容忍度。它决定了模型“学什么”（是否忽略异常值）。λ_1, λ_2 (Regularization)：控制模型的 Complexity（复杂度）和 Posterior Belief（后验信念）。它决定了模型怎么学（权重的分布形态）。

如果数据非常脏（含大量异常），σ 是主导因素。必须调小 σ 以拒绝学习异常。如果数据比较干净，但测试集包含很多没见过的新模式（Concept Drift），λ_1 是主导因素。必须精细调节 λ_1 以确保 Uncertainty 分数能准确反映模型的惊讶程度。

Loss 参数设计。

1. RevIN 的优势：由于使用了 RevIN，输入和输出的尺度都被标准化到了 $N(0,1)$ 附近。意味着权重的尺度是可控的。不需要为不同数据集大幅调整 λ。

参考文献

[1]